But this code also matches a line where the first name has a title and the second doesn't:

tut_regexp/attendee_validator_2.rb

```
re = %r{
  (?:(?<title>Mrs | Mr | Ms | Dr )\s)? (.*?)
  \s and \s
  (\g<title>\s)? (.+)
}x
match_data = re.match("Mr Smith and Sally")
match_data[0]       # => "Mr Smith and Sally"
match_data[:title]  # => "Mr"
```

We need to make the second test for a title mandatory if the first test matches. That's where the conditional subpatterns come in.

The syntax (?(n)subpattern) will apply the subpattern match only if a previous group number *n* also matched. You can also test named groups using the syntaxes (?(<name>)subpattern) or (?('name')subpattern).

In our case, we want to apply a test for the second title if the first title is present. That first title is matched by the group named title, so the condition group looks like (?<title>...):

tut_regexp/attendee_validator_3.rb

```
re = %r{
  (?:(?<title>Mrs | Mr | Ms | Dr )\s)? (.*?)
  \s and \s
  (?(<title>)\g<title>\s) (.+)
}x
match_data = re.match("Mr Smith and Sally")
match_data[0]       # => "Mr Smith and Sally"
match_data[:title]  # => nil
```

This didn't work—the match succeeded when we expected it to fail. That's because the regular expression applied *backtracking*. It matched the optional first name, the *and*, and then was told to match a second title (because group 1 matched the first). There's no second title, so the match failed. But rather than stopping, the engine went back to explore alternatives.

It noticed that the first title was optional, and so it tried matching the whole pattern again, this time skipping the title. It successfully matched *Mr Smith* using the (.*?) group, and matched *Sally* with the second name group. So we want to tell it never to backtrack over the first name—once it has found a title there, it has to use it. (?>...) to the rescue:

tut_regexp/attendee_validator_4.rb

```
re = %r{
  ^(?>
      (?:(?<title>Mrs | Mr | Ms | Dr )\s)? (.*?)
      \s and \s
    )
    (?(<title>)\g<title>\s) (.+)
}x
match_data = re.match("Mr Smith and Sally")
match_data                # => nil

successful_match = re.match("Mr Smith and Ms Sally")
successful_match[0]       # => "Mr Smith and Ms Sally"
successful_match[:title]  # => "Ms"
```